

< Open-Domain Question Answering Wrap-up Report >

팀명 : NLP_10 치맥회동

팀원 : 김광영_T8020, 박원규_T8079, 신지수_T8110, 유채영_T8128, 장수현_T8173, 한해별_T8217

프로젝트 기간 : 25. 12. 01 ~ 25. 12. 11

I. 프로젝트 개요

1) 목적/구성

- 사전에 구축된 **Knowledge resource** 에서 **질문(Query)**과 관련된 문서를 찾아서 **답변**하는 모델 개발
- **Two-stage** 구성
 - 1) 질문에 관련된 **문서를 찾아주는 Retriever**
 - 2) 문서를 읽고 적절한 **답변을 찾거나 만들어주는 Reader**
- 평가 방법은 Exact Match(**EM**), **F1 Score**

2) 개발환경

- OS : Linux #112-Ubuntu SMP
- GPU : Tesla V100-SXM2-32GB 3 대
- IDE : VS Code, Jupyter Notebook
- 협업 Tool : Github
- Pytorch : 2. 1. 0
- Pandas : 2. 3. 3
- Transformers : 4. 41. 0
- Tokenizers : 0. 19. 1
- scikit-learn : 1. 7. 2
- Random seed : 42

3) 데이터셋 구성

- Dataset 샘플 개수

Data set	샘플 수
train	3,952
validation	240
test	600
Wikipedia	60,613

- Train / Validation dataset 필드 구성

필드	설명	Dtype
title	context 의 주제	str
context	정답을 찾을 수 있는 원문	str

question	context 기반 질문	str
id	질문-답변 쌍 고유 ID	str
answers	{ answer_start : 정답 시작 index, text : 정답 }	dict
document_id	context 고유 ID	int64

- Test dataset 필드 구성

필드	설명	Dtype
question	context 기반 질문	str
id	질문-답변 쌍 고유 ID	str

- Wikipedia 필드 구성 **이외 필드는 중요하지 않은 정보(EX. None)로 판단하여 기재 생략*

필드	설명	Dtype
text	context 의 주제	str
title	정답을 찾을 수 있는 원문	str
document_id	text 고유 ID	int64

- Train / Validation 샘플 예시

title	context	question	
미국 상원	미국 상의원 또는 미국 상원(United ...	대통령을 포함한 미국의 행정부 견제권을 갖는 국가 기관은?	
인사조직관리	'근대적 경영학' 또는 '고전적 경영학'에서 ...	현대적 인사조직관리의 시발점이 된 책은?	
id	answers	document_id	__index_level_0__
mrc-1-000067	{'answer_start': [235], 'text': ['하원']}	18293	42
mrc-0-004397	{'answer_start': [212], 'text': ['《경영의 실제》']}	51638	2873

II. 수행 절차

- 1) 팀원별 담당 실험

팀원	담당 실험
김광영	Reader 성능 실험, 모듈화, Hyper Parameter Tuning
박원규	데이터 전처리 및 Reranker 모델 테스트
신지수	Retrieval 분리 및 성능평가, Hyper Parameter Tuning

유채영

BM25, Hybrid Retrieval, Soft voting Ensemble

장수현

DPR, Hybrid Retrieval

한해별

DPR, Hyper Parameter Tuning, Ensemble

2) 프로젝트 타임라인



III. 수행 결과

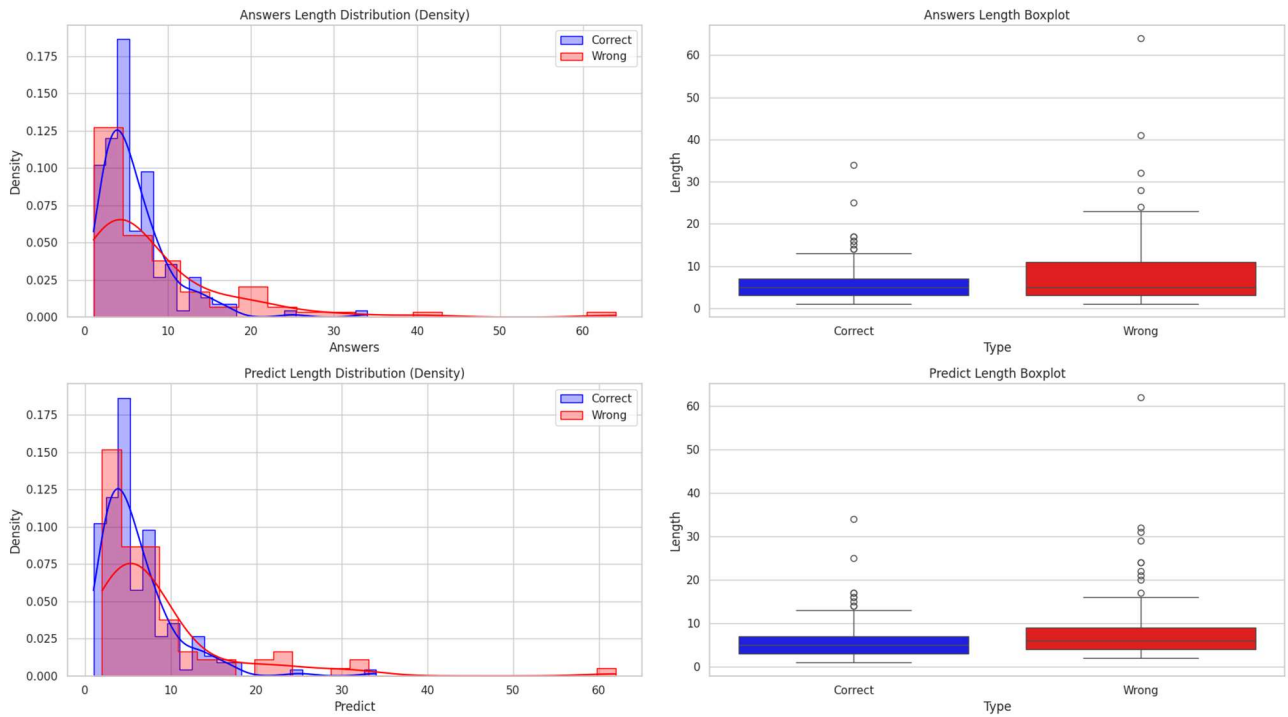
1) EDA

i. Baseline 기준 Correct, Wrong 특징 분석

- Baseline 모델(TF-IDF, klue/bert-base)의 EM 기준 정답 및 오답 데이터를 통해, 모델이 정답을 정확히 맞추기 어려운 데이터의 특징을 분석하였다.
- 정답 / 오답별 필드의 length **평균값, 최댓값**은 아래와 같다.

mean / max	Question	Answers	Predict	Text
Correct	30 / 59	6 / 34	6 / 34	892 / 2064
Wrong	27 / 52	9 / 64	9 / 62	955 / 2043

- Answers의 길이 분포와 유사하게 Predict 하지만, **Wrong case**에서는 Answer와 Prediction의 길이가 **Correct case**보다 긴 경향을 보인다. 이는 평가 기준(EM) 특성 상, Predict가 Answer와 한 글자라도 다른 경우에도 틀린 것으로 간주하기 때문으로 판단된다. Predict **Post-Processing**(후처리)이 필요해 보인다.



2) 데이터 전처리

i. Wikipedia 중복 샘플 제거

- text(위키피디아 문서의 원문)가 중복되는 샘플 3,876 개를 제거하여, 동일한 문서들이 문서 후보(top-k)에 오르지 않도록 했다.

	Before	After
Wikipedia	60,613	56,737

3) 데이터 증강

i. KorQuAD 기반 데이터 증강

- 기계 번역 데이터셋인 KorQuAD1.0, KorQuAD2.0 를 활용한 증강을 통해 모델의 강건성을 확보하고자 했다.
- 제공받은 원본 데이터 약 4000 개, KorQuAD1.0 약 60000 개, KorQuAD2.0 약 86000 개의 데이터로 구성되었다.
- KorQuAD1.0 과 KorQuAD2.0 의 컬럼값 및 컬럼 배치를 원본 데이터에 맞게 전처리 후 사용하였다. (SQuAD)
- 데이터의 개수가 크게 늘어나는 것을 고려하여, 학습 Epoch 를 조절하며 EM 과 F1 Score 를 확인했고, 동일 베이스라인 코드를 활용했다.
- 전체 데이터와 일부 샘플링 데이터(10000 개 / 2000 개) 등을 섞어 데이터셋을 구성했다.

데이터 비율	학습 Epoch	EM	F1
원본 데이터	5	30.4200%	43.9200%
원본 데이터 + KorQuAD1.0 + KorQuAD2.0	5	28.7500%	42.4500%
	10	28.9800%	42.6900%
	15	27.1300%	40.6600%
원본 데이터 + KorQuAD1.0	5	30.1700%	42.8800%
원본 데이터 + KorQuAD2.0	5	23.3300%	34.7500%
원본 데이터 + (KorQuAD1.0 + KorQuAD2.0) 10000 개 샘플링	5	30.3300%	43.8200%
원본 데이터 + (KorQuAD1.0 + KorQuAD2.0) 2000 개 샘플링	5	29.0800%	42.9600%

➔ KorQuAD1.0, KorQuAD2.0 을 활용해 증강한 모든 데이터셋이 원본 데이터셋에 비해 **낮은 성능을 보였다**. 이는 단순히 데이터셋을 합치는 전처리 뿐 아니라, 문제의 특성에 맞는 노이즈 제거 등의 전처리가 필요한 것으로 보인다. 또한 늘어난 데이터 규모에 따른 하이퍼파라미터 조정이 필요할 것으로 예상된다.

4) Retrieval

i. Sparse Retrieval

- 단어의 빈도(TF)와 역빈도(ITF)를 단순히 곱하는 **TF-IDF** 를 보정한 **BM25** 를 도입했다. 문서 길이와 단어 빈도를 보정한 가중치를 계산하여 Query 에 적합한 문서를 더 잘 찾을 수 있도록 했다.
- **Reader 모델별 Top-k(3~20) 최적값**을 찾기 위해 튜닝을 진행하였다. 튜닝 결과를 반영한 성능은 아래와 같다.

Retrieval	Reader	Top-k	Eval EM / F1	Test EM / F1
TF-IDF	klue/bert	10	-	34.17 / 42.88
	klue/bert	10	43.75 / 51.64	43.75 / 56.45
BM25	klue/bert	5	44.58 / 52.93	41.25 / 55.98
	roberta-large	10	57.50 / 65.55	55.50 67.75

➔ TF-IDF 대비 EM, F1 모두 **10% 이상 성능이 향상**되었다.

ii. Dense Retrieval

- **DPR**
- Dense Passage Retrieval(DPR)은 추가적인 사전학습 없이도 적은 수의 questions-passages 쌍만으로 고성능의 dense embedding 모델을 학습할 수 있도록 고안된 Retrieval 방식이다.

- DPR 은 질문과 문서를 각각 임베딩한 뒤 유사도로 문서를 찾는 dual-encoder 방식이며, Query Encoder, Passage Encoder 가 각각 필요하다. 본 프로젝트에서는 먼저 아래의 모델로 구현을 진행하였다.

모델명	Query encoder	Passage encoder	데이터셋
facebook/dpr-single-nq-base	dpr-question_encoder-single-nq-base	dpr-ctx_encoder-single-nq-base	NQ
facebook/dpr-multiset-base	dpr-question_encoder-multiset-base	dpr-ctx_encoder-multiset-base	NQ, TriviaQA, WebQuestions , CuratedTREC

- Facebook DPR 모델들은 한국어 데이터로 학습된 모델이 아니기 때문에 한국어 위키피디아 데이터셋을 사용하는 본 프로젝트에 직접 적용하기에는 한계가 있었다. 이에 따라 본 프로젝트에서는 한국어 환경에 맞는 Dense Retrieval 모델로 BGE-m3-ko 를 사용하여 DPR 형태의 검색을 구성하였다.

BGE-m3-ko 는 question/passage 를 각각 임베딩해 유사도로 Top-k 를 찾는 dual-encoder 방식으로, 의미적으로 유사한 문서를 검색하는 데 적합한 모델이다. 사전에 모든 위키 문서를 임베딩하여 인덱싱해 두고, 질의가 들어오면 질의 임베딩과의 유사도를 계산해 빠르게 상위 문서를 선택하는 방식으로, 이를 통해 sparse 기반 검색의 한계를 보완한다. DPR 의 사용으로 EM 은 약 1.2% 향상되었으나 부족하다고 느껴 Hybrid Retrieval 의 도입을 고려하게 되었다.

iii. Hybrid Retrieval

- 키워드 매칭(Sparse)과 의미 매칭(Dense)**의 이점을 모두 반영하기 위해 Hybrid Retrieval 을 도입했다.
- Reader 모델은 roberta-large 로, Dense 모델은 BGE-m3-ko 로 통일하였다.
- Hybrid 비율(alpha), Top-k(3~20) 최적값**을 찾기 위해 튜닝을 진행하였다. 튜닝 결과를 반영한 성능은 아래와 같다.

* Alpha = 0.0(Only BM25) ~ 1.0(Only BGE)

Retrieval	Top-k	Alpha	Eval EM / F1	Test EM / F1
BM25	10	-	57.50 / 65.55	55.50 / 67.75
	5	0.3	52.08 / 60.10	56.67 / 67.40
Hybrid	10	0.3	50.41 / 58.35	55.83 / 66.49
	10	0.5	51.25 / 59.02	57.08 67.66

➔ 단독 Retrieval 대비 **Eval**에서는 오히려 성능이 소폭 하락하였고, **Test**에서는 **EM**이 약 2% 상승했다.

Question 과 Answers 가 모두 문서에서 추출된 데이터셋의 특성 상, Dense 의 이점인 **의미적 유사도 매칭**이 **EM**에 잘 반영되지 않는 것으로 판단된다.

- Hybrid Retrieval 성능 향상을 위해 1) **Dense 대조 학습** 2) **BM25 Tokenizer 변경**을 했다.
 - BM25 문서 후보 중 정답과 유사하지 않은 문서를 Hard Negative 로 두어, Dense 모델이 벡터 공간에서 **Question**을 기준으로 **Positive(정답)** 문서는 가깝게, **Negative** 문서는 멀게 위치시키도록 fine-tuned 했다.
EX) sample = (Q, Positive, Negative1, Negative2)

Retrieval	Top-k	Alpha	Eval EM / F1	Test EM / F1
BGE	10	-	48.75 / 56.23	51.67 / 63.49

BGE(fine-tuned)	10	-	50.83 / 57.84	52.08 / 64.73
Hybrid	10	0.5	51.25 / 59.02	57.08 / 67.66
Hybrid(fine-tuned)	5	0.3	59.58 / 67.23	63.75 / 74.08

- 2) 기존 토큰라이저(roberta-large)는 일부 토큰의 접두사에 '##'이 붙어, 키워드 매칭을 저하시킨다고 판단하여, **mecab 라이브러리의 형태소 분리기**를 BM25 토큰라이저로 사용하였다.

Retrieval	tokenizer	Top-k	Alpha	Eval EM / F1	Test EM / F1
Hybrid (fine-tuned)	roberta	5	0.3	59.58 / 67.23	63.75 / 74.08
	Mecab (명사분리)	5	0.3	-	62.92 / 73.95
Hybrid (fine-tuned)	Mecab (형태소분리)	5	0.3	59.58 / 67.31	64.17 / 75.33
		3	0.3	60.00 / 67.69	65.83 / 77.49

- 대조학습의 경우, 기존 대비 성능이 향상되었으며 특히 **Hybrid에 적용했을 때 EM이 큰 폭으로 상승했다**. 토큰라이저의 경우, 명사만을 토큰화 했을 때보다 **모든 형태소를 토큰화 했을 때 더 좋은 성능**을 보였다. 이는 명사를 추출하는 과정에서 동사가 탈락한 결과로 판단된다.

5) Reranker

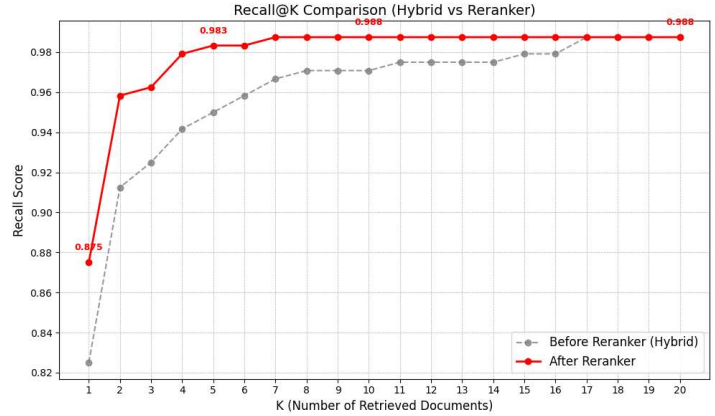
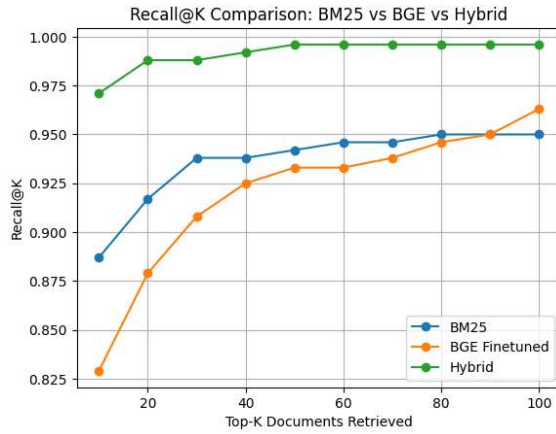
i. 모델 테스트

- 기학습된 Reranker의 효용성을 파악하기 위해 BAAI/bge-reranker-v2-m3의 적용 여부에 따른 Baseline Code의 성능을 확인하였다.

	Top-k	reranking	Eval EM / F1	Test EM / F1
Baseline	10	-	32.92 / 44.76	35.56 / 47.51
Baseline + Reranker	50	3	40.83 / 53.95	42.78 / 54.05

- 테스트 결과, 단순 Retrieval 기반으로 10개의 문서를 Reader에 입력하는 Baseline 대비, Reranker를 활용하여 50개의 문서 중 상위 3개를 재선정해 Reader에 전달하는 방식이 EM 및 F1 성능을 확실하게 개선하는 것으로 확인되었다. (약 20~25% 가량 향상)

6) 최종 Retrieval 성능 평가 (Recall@K)



7) Reader

i. 실험 개요 및 설정

본 실험은 ODQA(Open-Domain Question Answering) 시스템 내 Reader 모델의 성능 최적화를 목적으로 수행되었다. 실험의 변인 통제를 위해 Retriever 모델은 Baseline 코드(TF-IDF)로 고정하였으며, 다양한 Pre-trained LM(Language Model)을 교체하며 성능 변화를 관찰하였다.

- 평가 지표: Exact Match (EM) 및 F1 Score (소수점 셋째 자리에서 반올림)
- 비교: Baseline 이외의 전처리(Token 99% cut 등) 적용 시 비교란에 별도 표기함.

ii. 실험 결과

모델명	Eval EM	Eval F1	제출 EM	제출 F1	비고
uomnf97/klue-roberta-finetuned-korquad-v2	72.91	80.93	42.50	55.03	토큰 기준 99% 컷
klue/roberta-large	68.75	75.36	41.67	53.73	
klue/roberta-large	67.91	76.81	-	-	토큰 기준 99% 컷
uomnf97/klue-roberta-finetuned-korquad-v2	67.08	75.52	-	-	
monologg/kobigbird-bert-base	61.66	71.28	40.42	53.48	
monologg/koelectra-base-v3-finetuned-korquad	61.25	68.94	-	-	
klue/bert-base	-	-	34.17	42.88	
xlm-roberta-large	52.08	65.03	-	-	
monologg/koelectra-base-v3-discriminator	51.66	61.29	-	-	
team-lucid/deberta-v3-xlarge-korean	48.75	60.31	-	-	
kykim/bert-kor-base	40.41	51.76	-	-	
answerdotai/ModernBERT-base	22.91	28.84	-	-	한국어 학습 X

iii. 성능 분석

■ 정량적 분석

1. Baseline 대비 성능 향상: 단순히 Reader 모델을 klue/bert-base 에서 RoBERTa-large 계열로 변경하는 것만으로도 약 6%~8%의 EM 성능 향상을 확인하였다.
2. Model Capacity: 비교적 파라미터 수가 많은 Large 모델들이 Base 모델 대비 높은 성능을 보였으며, 한국어 학습이 부족한 모델(MoERNBERT 등)은 최신 아키텍처임에도 성능이 저조했다.

■ 성능 우수 원인에 대한 이론적 고찰

상위권을 기록한 RoBERTa-large 기반 모델들이 우수한 성능을 보인 이유는 다음과 같은 이론적 근거로 해석된다

1. RoBERTa 아키텍처의 강점 (Robustly Optimized BERT Approach)

NSP 제거 및 Dynamic Masking: RoBERTa 논문(Liu et al., 2019)에 따르면, BERT 의 Next Sentence Prediction(NSP) 과제는 다운스트림 태스크 성능에 부정적 영향을 줄 수 있다. 이를 제거하고 학습 데이터를 복제해 매번 다른 마스킹을 적용하는 Dynamic Masking 을 통해 문맥 이해도를 극대화한 것이 QA 태스크에서 유효하게 작용하였다.
2. Intermediate Fine-tuning (중간 단계 학습) 효과

1 위를 기록한 모델(uomnf97/...)은 KorQuAD v2 데이터셋으로 Intermediate Fine-tuning 을 거친 모델이다. 관련 연구(Phang et al., 2020)에 따르면, 타겟 태스크와 유사한 대규모 데이터셋으로 중간 학습을 수행할 경우 모델이 QA 구조를 사전에 학습하여 적은 데이터에서도 높은 성능과 빠른 수렴을 보인다.
3. 한국어 특화 토큰라이저 (KLUE Benchmark)

다국어 모델(XLM-RoBERTa) 대비 KLUE-RoBERTa 가 압도적인 성능을 보였다. 이는 한국어의 교착어적 특성을 반영한 Morpheme-aware Subword Tokenizer 의 적용 유무가 결정적이었으며, 한국어의 문법적/의미적 모호성을 해소하는 데 더 효과적이었음을 시사한다.

iv. 한계점 및 결론

실험 결과, "Large Scale 모델 + 한국어 특화 + Intermediate Fine-tuning"이 적용된 모델이 최적의 성능을 보임을 확인하였다.

그러나 성능의 상한선은 모델이 처리 가능한 입력 토큰 길이(Max Sequence Length)의 한계와 직결되어 있다. 긴 문서를 처리하기 위해 슬라이싱(Slicing)을 적용할 경우, 입력 토큰의 의미 단위가 해제되거나 문맥이 단절(Context Fragmentation)되는 구조적 문제가 발생함을 확인하였다.

8) 하이퍼파라미터 튜닝

i. monologg/kobigbird-bert-base

- 큰 모델이라 OOM 이슈로 배치사이즈는 테스트하지 못하였고, Reader 실험 도중 overfitting 이슈를 확인해 train-epochs 를 적게 조절하였다.

Parameter	Values	최적값
learning_rate	1e-6 ~ 5e-5	3e-5
num_train_epochs	1~5	3
weight_decay	0.0, 0.01	0.01

warmup_ratio	0.0, 0.05, 0.1	0.0
--------------	----------------	------------

ii. monologg/koelectra-base-v3-finetuned-korquad\

Parameter	Values	최적값
		0.00002274
learning_rate	1e-6 ~ 5e-5	
num_train_epochs	1~3	2
weight_decay	[0.0, 0.1]	0.08074
warmup_ratio	[0.0, 1]	0.1565

iii. klue/roberta-large

- **Bayesian Optimization** 기반 Exact Match 를 최대화하는 방향으로 **10 회의 실험**을 진행하였다.

Parameter	Values	최적값
learning_rate	1e-6 ~ 5e-5	8.556e-6
per_device_train_batch_size	8, 16	16
num_train_epochs	3, 4, 5	4
weight_decay	0.0, 0.01, 0.1	0.01
warmup_ratio	0.0, 0.05, 0.1	0.05

iv. klue-roberta-finetuned-korquad-v2

- 기본 설정: earlystopping_patience 2, earlystopping_threshold 0.0, per_device_train_batch_size 8
- Exact Match 를 최대화하는 방향으로 총 20 회의 실험을 진행하였다.

Parameter	Values	최적값
learning_rate	1e-6 ~ 5e-5	3e-6
num_train_epochs	10	5
weight_decay	0.0, 0.01	0.0
warmup_ratio	0.0, 0.05, 0.1	0.0

v. 하이퍼파라미터 튜닝 결과

MRC	Train EM / F1	Test EM / F1
kobigbird	71.25 / 79.69	60.00 / 71.92
kobigbird (k-fold)	측정하지 않음	51.25 / 66.60
roberta_v2	74.16 / 81.98	72.08 / 84.47
roberta	72.5 / 80.34	69.17 / 79.61

9) 앙상블

i. Hard voting

- 7)에서 하이퍼파라미터 튜닝을 진행한 3 개의 결과 파일과 Retrieval 과 Reranker 에서 하이퍼파라미터를 달리하여 실험한 상위 7 개의 결과 파일을 앙상블하였다.
- Reader: 3 개, Reranker: 2 개, Retrieval: 5 개

Reader	Retrieval	Reranker	Public EM / F1	Private EM / F1
kobigbird	tokenizer			
roberta_v2	top-k	top-k	74.17 / 84.43	70.00 / 81.22
roberta	alpha			

ii. Soft voting (Weighted voting)

- 하이퍼파라미터 튜닝을 통해 찾은 **최적의 값으로 학습한 MRC 의 예측 값**으로 앙상블을 진행했다.
- Question 별 **20 개의 답변 후보 확률을 각각 누적**하여, 최고치를 최종 답변으로 제출했다.

kobigbird	kobigbird (k-fold)	roberta_v2	roberta	Test EM / F1
1	1	1	1	67.50 / 81.00
0.5	-	2	1.5	74.17 / 84.43
-	-	2	1.5	74.17 / 84.32

10) 데이터 후처리

- Reader 모델이 예측한 정답의 포맷을 통일하고, 불필요한 조사나 특수문자를 제거하여 채점 기준(Exact Match)에 부합하도록 LLM 기반의 후처리 파이프라인을 구축하였다.

1. LLM 기반 정규화 모델

- 사용 모델: Qwen/Qwen2.5-7B-Instruct
- 선정 이유: 한국어 및 다국어 처리 능력이 우수하고 지시문 이행 능력이 뛰어난 7B 사이즈 모델 선정
- 환경 설정: 자원 효율성을 위해 4-bit Quantization 을 적용하여 메모리 사용량 최소화
- 생성 파라미터: Temperature=0.1 로 설정하여 창의성을 낮게하고 사실 기반의 일관된 답변 유도

2. 프롬프트 엔지니어링

- 단순한 정제가 아닌, 질문(Question)과 지문(Context)을 함께 입력하여 문맥에 맞는 '핵심 명사'를 추출하도록 유도했다. 특히 한국어 ODQA 의 특성상 조사가 포함되면 오답 처리될 확률이 높으므로, 이를 제거하는 데 집중했다

3. 하이브리드 정제 파이프 라인

- LLM 의 환각(Hallucination)이나 과도한 변형을 방지하기 위해, Rule-based 와 Model-based 방식을 결합한 3 단계 프로세스를 적용하였다.
- 1 단계 (Rule-based): 정규표현식(Regex)을 사용하여 괄호, 특수문자, 공백 등 기본적인 노이즈를 1 차적으로 제거.
- 2 단계 (LLM Refinement): 1 차 정제된 값을 LLM 에 입력하여 문맥에 맞게 최종 교정 수행.
- 3 단계 (Safety Check): LLM 이 생성한 답변이 원본 예측값보다 지나치게 길어지거나(2 글자 이상 초과), 빈 값을 반환할 경우 오류로 간주하여 원본 예측값(Rule-based 결과)을 유지하는 안전장치(Fallback) 적용.

4. 후처리 실험 결과 및 분석

실험명	제출 EM	제출 F1	비고
Baseline	60.00	71.92	
LLm Post-Processing	28.33	64.71	LLm 후처리 적용

1. 정량적 성능 하락:

- 기대와 달리 LLM 후처리를 적용했을 때 Eval EM 기준 약 7.2%, 제출(Submission) EM 기준 약 31.6%의 대폭적인 성능 하락이 발생하였다.
- 특히 제출 스코어에서의 하락 폭이 매우 큰 것으로 보아, 리더보드의 채점 데이터셋(Test Set)에 포함된 정답 형식과 LLM 이 생성한 답변 스타일 간의 괴리가 컸음을 시사한다.

2. 원인 분석 (Failure Analysis):

- 과잉 교정 (Over-Correction): LLM 이 문맥을 이해하여 답을 재생성하는 과정에서, 정답으로 인정될 수 있는 파생어까지 불필요하게 변경하거나, 핵심 단어를 다른 유의어로 교체하여 Exact Match 점수가 하락한 것으로 분석된다.
- 포맷 불일치: "가장 짧은 핵심 명사"를 요구했음에도 불구하고, LLM 이 문장형으로 답하거나 답변의 뉘앙스를 미세하게 변경하여 채점 기준을 벗어나는 케이스가 발생하였다.
- 안전장치(Fallback)의 한계: 원본 길이 대비 2 글자 이상 길어질 경우 원본을 사용하는 로직을 적용했으나, 길이가 비슷하면서 내용이 바뀌는 오답(Hallucination) 케이스를 완벽히 필터링하지 못한 것으로 보인다.

본 실험을 통해 ODQA 태스크, 특히 Exact Match 가 중요한 평가 지표인 환경에서는 LLM 의 생성 능력보다는 Rule-based 방식의 보수적인 전처리(특수문자 제거, 공백 처리 등)가 성능 유지에 더 유리함을 확인하였다. 향후 LLM 을 도입할 경우, 답변을 직접 생성하기보다는 정답 후보군(Candidate) 중에서 선택하는 Reranker 역할로 활용하는 것이 더 적합할 것으로 판단된다.

IV. 최종 성능 (Public 6 등, Private 3 등)

	Public EM	PublicF1	Private EM	Private F1
baseline	34.17	42.88	36.67	48.70
nlp_10	74.17	84.43	70.00	81.22

순위	팀이름	팀멤버	EM	F1	제출횟수	최종 제출
내등수 3	NLP_10조		70.0000%	81.2200%	42	1h
1	NLP_02조 		74.1700%	84.5000%	23	9m

V. 평가 및 회고

1) 팀 회고

- 잘한 점 😊
 - 1) 첫 팀 프로젝트를 나름 성공리(최종 3 등)에 마무리하였다!
- 아쉬운 점 😞
 - 1) Github 이나 jira 와 같은 협업 Tool 활용이 적었다.
- 개선할 점 📌
 - 1) 프로젝트 시작 초반에 각자 과제, baseline 이해하고 하고 싶은 실험 생각하기
 - 2) Github 협업 규칙 정하기
 - 3) 미사용 중인 서버 활용할 수 있도록 서버 현황 공유 페이지 만들기

2) 개인 회고

- 김광영
- 잘한 점 😊
 1. 이번 프로젝트에서는 크게 없다고 생각한다.
- 아쉬운 점 😞
 1. 모듈화를 시도하다가 시간에 쫓겨 중간에 포기하고 막 진행함
 2. LLM postprocessing 도 시도하다 시간에 쫓겨 중간에 포기하고 생략
 3. 협업툴은 slack 정도.. 소통으로 사용함
 4. 목표 EM75%를 달성하지 못함
- 개선할 점 📌
 1. Baseline 을 개선 및 알아보기 쉽게 고쳐서 그걸 기반으로 실험을 진행하는게 좋을 듯
 2. 1 번을 지키면 github 를 더 쉽게 사용할 수 있지 않을까
 3. 후반에 시간에 쫓겨도 기록을 계속 남겨야한다. (실험 관리의 중요성)
- 박원규
- 잘한 점 😊
 - 1) 서로 다른 양식의 데이터셋을 전처리하고 병합하는 Task 를 수행했다.
 - 2) 프로젝트의 구조를 파악하고 아이디어를 떠올리는 과정을 경험했다.
- 아쉬운 점 😞
 - 1) 아이디어가 구체적으로 실행되지 못했다.
 - 2) 데이터 증강을 통한 팀 내 기여를 실현하지 못했다.
 - 3) 스스로 생각하기에 시간 관리 및 성실성 문제가 있었다고 판단된다
 - 4) 데이터 중심의 사고방식에서 벗어나지 못했다. → 모델 및 학습 기법 등으로 지식 확장이 필요하다..
- 개선할 점 📌
 - 1) 단순 형식 전처리가 아닌 노이즈/결측치 처리 등 프로젝트에 맞는 전처리 수행이 필요하다.
 - 2) 스스로에게 더욱 엄격한 자기관리가 필요할 것 같다. 반성이 필요하다.
 - 3) 여러 모델의 특성 및 최신 기술 동향, 앙상블 등의 기법에 대한 깊은 이해가 필요하다.
 - 4) 규칙적인 생활 및 시간관리를 통해 효율적인 협업이 이루어질 수 있도록 해야한다.
- 신지수

- 잘한 점 😊
 - 1) 프로젝트 코드를 이해하고 다양한 방법으로 만져보았다.
- 아쉬운 점 😞
 - 1) 협업 컨벤션이나 구조를 처음부터 정하고 했으면 좀 더 체계적으로 기록할 수 있었을 것 같다.
 - 2) 시간 부족으로 LLM 의존적으로 코드를 작성했다.
- 개선할 점 📌
 - 1) 스스로 이해하는 선에서 코드를 작성해보자
 - 2) 협업 방식 초반에 정리하기
 - 3) 실험방향 스스로 생각해보기
- 유채영
- 잘한 점 😊
 - 1) 이전 프로젝트보다 프로젝트 일정 관리, 실험 관리가 나아졌다. 덕분에 구상한 아이디어의 대부분을 실험해 볼 수 있었다.
 - 2) 처음 AI IDE(Cursor)를 사용했다. 덕분에 baseline 코드를 이해하고, 리팩토링 하는 데 최소한의 시간을 투자하였다.
- 아쉬운 점 😞
 - 1) Git 활용 능력이 아직 서툴러서 원활한 협업이 안 되어서 아쉽다.
 - 2) 서버를 자꾸 날려 먹어서 나와 팀원의 실험을 방해했다.
 - 3) 코드 레벨에서의 성과에 몰입하여, 이론 공부가 부족했다.
 - 4) LLM 이 대부분의 코드를 작성했다.
- 개선할 점 📌
 - 1) Git, AI IDE, 서버 공부하기
 - 2) 프로젝트 초반에 이론 공부 탄탄히 해두기
 - 3) LLM 에게 질문하기 전 스스로 생각해보고, 적어도 LLM 의 답변을 복붙하지는 말기
- 장수현
- 잘한 점 😊
 - 1) MRC 의 동작 과정을 익힐 수 있었고, 팀 프로젝트를 통해 협업의 중요성과 필요성을 배웠다.
- 아쉬운 점 😞
 - 1) Github 나 Wandb 를 능숙하게 사용하지 못한 것 같다.
 - 2) 실험을 하던 것 들이 좋지 않은 결과가 자주나와 혼란스러웠다.
 - 3) 서버의 한계로 다양한 실험을 시도해보지 못한 것이 아쉽다.
- 개선할 점 📌
 - 1) 프로젝트를 시작하기전 해당 내용에 대해 가능한 완전하게 이해를 하고 시작하기.
 - 2) 학습이 돌아가는 시간을 잘 활용하기.
- 한해별
- 잘한 점 😊
 - 1) GitHub 와 wandb 를 활용하여 실험 관리 및 결과 추적을 시도해 보았다.
- 아쉬운 점 😞
 - 1) Baseline 코드를 충분히 이해하지 못한 상태에서 프로젝트를 진행하여, 전반적인 프로젝트 수행이 원활하지 않았다.
 - 2) 협업을 위한 내용 정리 및 GitHub, wandb 활용이 체계적으로 이루어지지 못했다.
 - 3) 강의 학습과 프로젝트 진행을 병행하지 못해 이론적 이해가 부족한 상태로 실습 위주로 진행하게 되었다.
 - 4) 서버에서만 코드를 돌려 많은 시도를 해보지 못했다.
- 개선할 점 📌
 - 1) Baseline 코드를 정확히 이해한 후 이를 바탕으로 리팩토링을 진행하여, 코드 수정과 실험 확장이 원활하도록

개선해야 한다.

- 2) 진행한 실험 및 결과에 대해 꾸준히 정리하고, GitHub 와 W&B 를 활용한 공유 및 협업 방식에 대해 추가로 학습할 필요가 있다.
- 3) 프로젝트 초반에 관련 강의를 먼저 수강하여, 이론적 기반을 충분히 다진 후 프로젝트를 진행해야 한다.
- 4) 로컬에서도 프로젝트를 진행할 수 있도록 환경을 만들어두어야 한다.